```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, roc_auc_score, roc_curve
from sklearn.preprocessing import label_binarize
import pickle
import matplotlib.pyplot as plt
from itertools import cycle

from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
# Load the dataset from Google Drive
data =
pd.read_csv('/content/drive/MyDrive/par/pe_section_headers.csv')

data
```


```
{"summary":"{\n  \"name\": \"data\",\n  \"rows\": 43293,\n
\"fields\": [\n    {\n      \"column\": \"hash\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 43144,\n        \"samples\": [\n
\"a7bce79b0bfe09cc6ecc2df1a2026180\",\n
\"951714ce4388bfd415c82fa4fbb9163e\",\n
\"5e66af939db0473421272095b1199466\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"size_of_data\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1173483,\n        \"min\": 0,\n        \"max\": 52962816,\n
\"num_unique_values\": 2659,\n        \"samples\": [\n
3246080,\n          1959424,\n          1450496\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"virtual_address\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
83684,\n        \"min\": 4096,\n        \"max\": 13152256,\n
\"num_unique_values\": 129,\n        \"samples\": [\n
692224,\n          659456,\n          376832\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"entropy\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.9069034159965675,\n        \"min\": 0.0,\n        \"max\":
7.999965184067571,\n        \"num_unique_values\": 17613,\n
\"samples\": [\n          6.638889910844765,\n
6.5874033654282,\n          5.544255514252651\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"virtual_size\",\n
```

\"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 1187789,\n   \"min\": 5,\n   \"max\": 52962523,\n   \"num_unique_values\": 11715,\n   \"samples\": [\n   605767,\n   215540,\n   80693\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"malware\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\": 0,\n   \"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   0,\n   1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"data"}

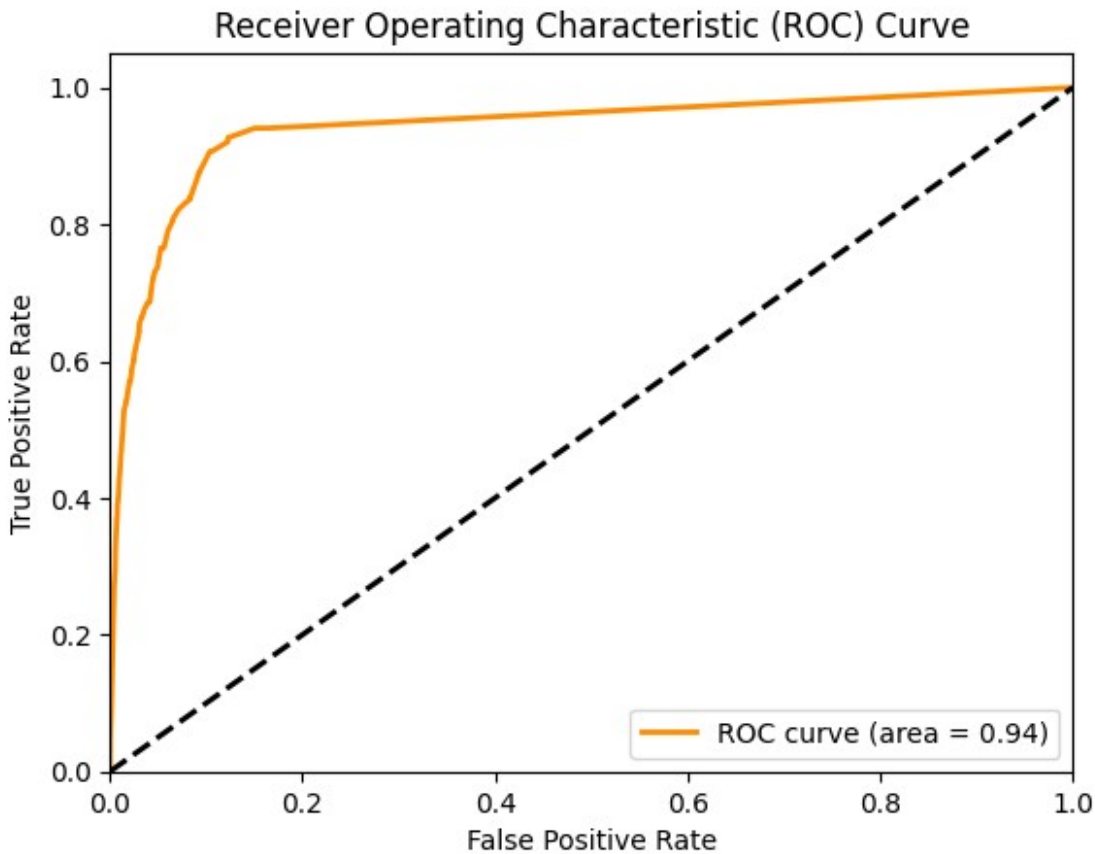
```
# Prepare the data
X = data[[ 'size_of_data', 'virtual_address', 'entropy',
'virtual_size']]
y = data['malware']

# Encode the target variable
y_encoded, y_classes = pd.factorize(y)

# Binarize the output for multi-class ROC AUC
n_classes = len(y_classes)
y_bin = label_binarize(y_encoded, classes=range(n_classes))

# Split the dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
test_size=0.2, random_state=42)

# Train Random Forest Classifier
rfc = RandomForestClassifier(n_estimators=100, random_state=42)
rfc.fit(X_train, y_train)
```

```
RandomForestClassifier(random_state=42)
```

```
# Save the model using pickle
with open('pe_section_header.pkl', 'wb') as file:
    pickle.dump(rfc, file)

# Evaluate the model
# Evaluate the model
y_pred = rfc.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

y_classes_str=y_classes.astype(str).tolist()
classification_metrics = classification_report(y_test, y_pred,
target_names=y_classes_str)
print("Classification Report:\n", classification_metrics)

conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)
```

```python
# Calculate ROC and AUC
y_proba = rfc.predict_proba(X_test)
#y_test_bin = label_binarize(y_test, classes=range(n_classes))
roc_auc = roc_auc_score(y_test, y_proba[:, 1], multi_class='ovr')
print("ROC AUC Score:", roc_auc)

# Plot ROC curve
fpr, tpr, _ = roc_curve(y_test, y_proba[:, 1]) # Use the probabilities
for the positive class
roc_auc = roc_auc_score(y_test, y_proba[:, 1])


plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area =
{roc_auc:0.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

```
Accuracy: 0.9672017553990068
Classification Report:
              precision    recall  f1-score   support

           1       0.98      0.99      0.98      8305
           0       0.66      0.41      0.51       354

    accuracy                           0.97      8659
   macro avg       0.82      0.70      0.74      8659
weighted avg       0.96      0.97      0.96      8659

Confusion Matrix:
 [[8230   75]
 [ 209  145]]
ROC AUC Score: 0.9395680908308588
```




```
import warnings
warnings.filterwarnings("ignore")

size_of_data= int(input("Enter size_of_data: "))
virtual_address= int(input("Enter virtual_address: "))
entropy= float(input("Enter entropy: "))
virtual_size= int(input("Enter virtual_size: "))


# Make predictions
input_data = [[size_of_data, virtual_address, entropy, virtual_size]]
# Load the correct file name
with open('pe_section_header.pkl', 'rb') as file: # Check this file
name! It might be different
    loaded_model = pickle.load(file)

prediction = loaded_model.predict(input_data)
predicted_class = y_classes[prediction[0]]
print("Predicted target:", predicted_class)

Enter size_of_data: 26112
Enter virtual_address: 4096
Enter entropy: 6.33361
```

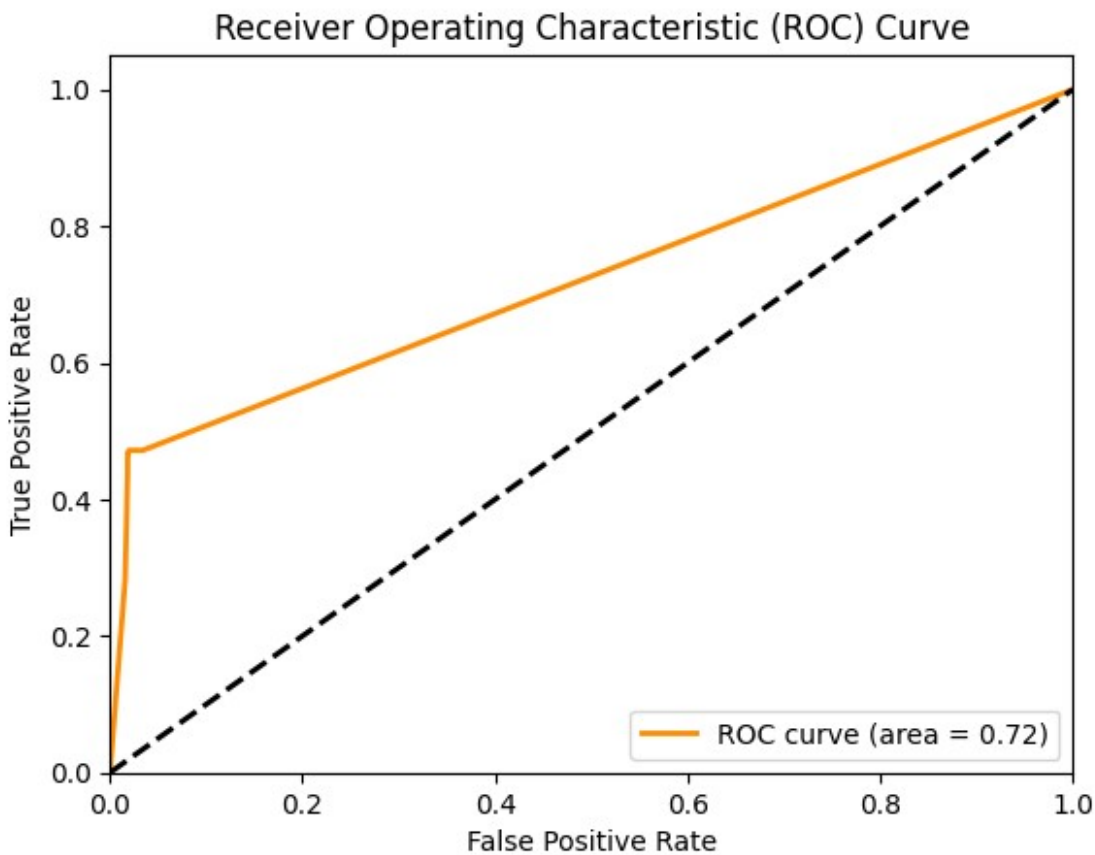
```
Enter virtual_size: 25628
Predicted target: 0
```

```
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier(criterion='gini', random_state=0)
dtc.fit(X_train, y_train)
```

```
DecisionTreeClassifier(random_state=0)
```

```
with open('pe_section_headers.pkl', 'wb') as file:
    pickle.dump(dtc, file)

# Evaluate the model
y_pred = dtc.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

y_classes_str=y_classes.astype(str).tolist()
classification_metrics = classification_report(y_test, y_pred,
target_names=y_classes_str)
print("Classification Report:\n", classification_metrics)

conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)


# Calculate ROC and AUC
y_proba = dtc.predict_proba(X_test)
#y_test_bin = label_binarize(y_test, classes=range(n_classes))
roc_auc = roc_auc_score(y_test, y_proba[:, 1], multi_class='ovr')
print("ROC AUC Score:", roc_auc)

# Plot ROC curve
fpr, tpr, _ = roc_curve(y_test, y_proba[:, 1]) # Use the probabilities
for the positive class
roc_auc = roc_auc_score(y_test, y_proba[:, 1])


plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area =
{roc_auc:0.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

```
Accuracy: 0.959926088462871
Classification Report:
               precision    recall  f1-score   support

           1       0.98      0.98      0.98      8305
           0       0.51      0.45      0.48       354

    accuracy                           0.96      8659
   macro avg       0.74      0.72      0.73      8659
weighted avg       0.96      0.96      0.96      8659

Confusion Matrix:
 [[8152  153]
 [ 194  160]]
ROC AUC Score: 0.7213650139287135
```




```python
import warnings
warnings.filterwarnings("ignore")

size_of_data= int(input("Enter size_of_data: "))
virtual_address= int(input("Enter virtual_address: "))
entropy= float(input("Enter entropy: "))
virtual_size= int(input("Enter virtual_size: "))
```

```
# Make predictions
input_data = [[size_of_data, virtual_address, entropy, virtual_size]]
# Load the correct file name
with open('pe_section_headers.pkl', 'rb') as file: # Changed file name
    loaded_model = pickle.load(file)

prediction = loaded_model.predict(input_data)
predicted_class = y_classes[prediction[0]]
print("Predicted target:", predicted_class)
```

```
Enter size_of_data: 16384
Enter virtual_address: 4096
Enter entropy: 5.558912
Enter virtual_size: 15876
Predicted target: 1
```

```
if(predicted_class==1):
  print("Malware")
else:
  print("Goodware")
```

```
Malware
```

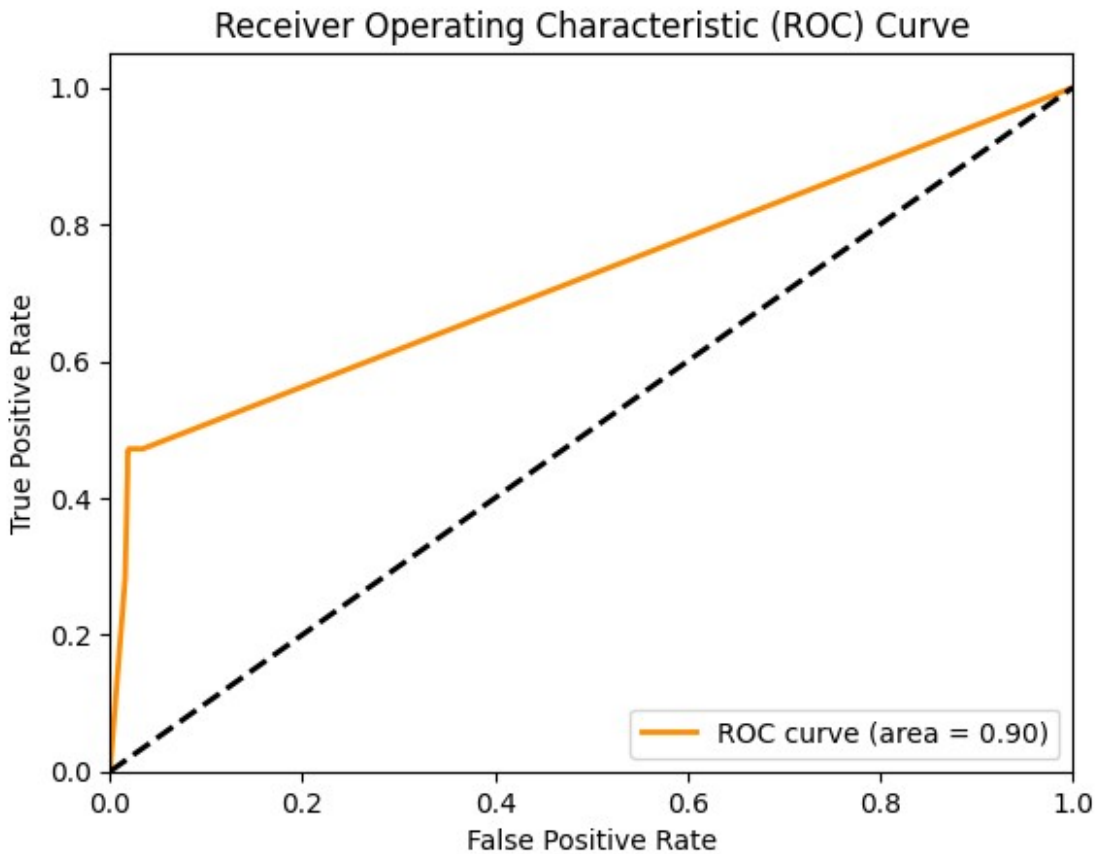
```
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier()
gb.fit(X_train , y_train)
```

```
GradientBoostingClassifier()
```

```
with open('pe_section_headers.pkl', 'wb') as file:
    pickle.dump(gb, file)

# Evaluate the model
y_pred = gb.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

y_classes_str=y_classes.astype(str).tolist()
classification_metrics = classification_report(y_test, y_pred,
target_names=y_classes_str)
print("Classification Report:\n", classification_metrics)

conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)


fpr, tpr, _ = roc_curve(y_test, y_proba[:, 1]) # Use the probabilities
for the positive class
roc_auc = roc_auc_score(y_test, y_proba[:, 1])
```

```python
# Calculate ROC and AUC
y_proba = gb.predict_proba(X_test)
#y_test_bin = label_binarize(y_test, classes=range(n_classes))
roc_auc = roc_auc_score(y_test, y_proba[:, 1], multi_class='ovr')
print("ROC AUC Score:", roc_auc)

# Plot ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area =
{roc_auc:0.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

```
Accuracy: 0.9645455595334335
Classification Report:
               precision    recall  f1-score   support

           1       0.97      1.00      0.98      8305
           0       0.69      0.25      0.36       354

    accuracy                           0.96      8659
   macro avg       0.83      0.62      0.67      8659
weighted avg       0.96      0.96      0.96      8659

Confusion Matrix:
 [[8265   40]
 [ 267   87]]
ROC AUC Score: 0.9028998595223761
```




```
import warnings
warnings.filterwarnings("ignore")

size_of_data= int(input("Enter size_of_data: "))
virtual_address= int(input("Enter virtual_address: "))
entropy= float(input("Enter entropy: "))
virtual_size= int(input("Enter virtual_size: "))


# Make predictions
input_data = [[size_of_data, virtual_address, entropy, virtual_size]]
# Load the correct file name
with open('pe_section_headers.pkl', 'rb') as file: # Changed file name
    loaded_model = pickle.load(file)

prediction = loaded_model.predict(input_data)
predicted_class = y_classes[prediction[0]]
print("Predicted target:", predicted_class)

Enter size_of_data: 23040
Enter virtual_address: 4096
Enter entropy: 6.4331
```

```
Enter virtual_size: 22738
Predicted target: 1
```

```
if(predicted_class==1):
  print("Malware")
else:
  print("Goodware")
```

```
Malware
```

```
from sklearn.ensemble import AdaBoostClassifier
ab = AdaBoostClassifier()
ab.fit(X_train , y_train)
```

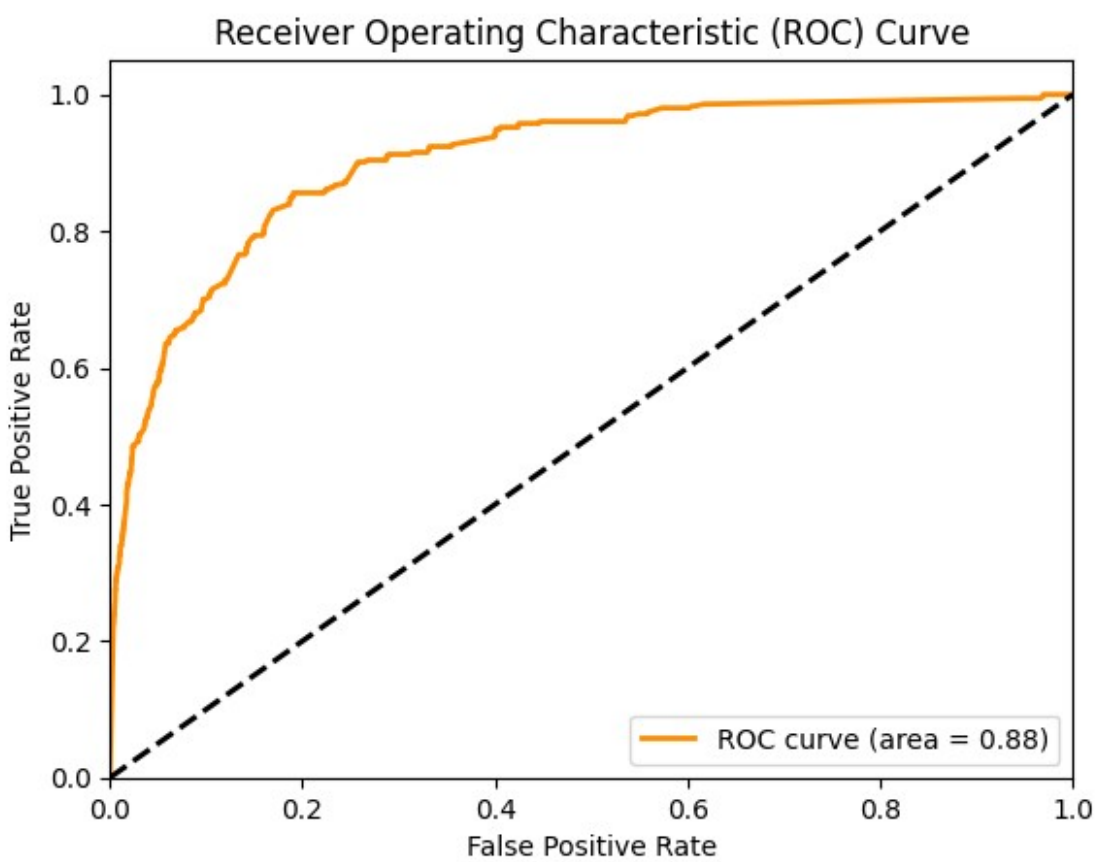
```
AdaBoostClassifier()
```

```
with open('pe_section_headers.pkl', 'wb') as file:
    pickle.dump(ab, file)
```

```
# Evaluate the model
y_pred = ab.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

y_classes_str=y_classes.astype(str).tolist()
classification_metrics = classification_report(y_test, y_pred,
target_names=y_classes_str)
print("Classification Report:\n", classification_metrics)

conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)

fpr, tpr, _ = roc_curve(y_test, y_proba[:, 1]) # Use the probabilities
for the positive class
roc_auc = roc_auc_score(y_test, y_proba[:, 1])


# Calculate ROC and AUC
y_proba = ab.predict_proba(X_test)
#y_test_bin = label_binarize(y_test, classes=range(n_classes))
roc_auc = roc_auc_score(y_test, y_proba[:, 1], multi_class='ovr')
print("ROC AUC Score:", roc_auc)

# Plot ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area =
{roc_auc:0.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

```
Accuracy: 0.9600415752396351
Classification Report:
              precision    recall  f1-score   support

           1       0.96      1.00      0.98      8305
           0       0.60      0.07      0.13       354

    accuracy                           0.96      8659
   macro avg       0.78      0.53      0.55      8659
weighted avg       0.95      0.96      0.94      8659

Confusion Matrix:
 [[8288   17]
 [ 329   25]]
ROC AUC Score: 0.8767358170321466
```




```
import warnings
warnings.filterwarnings("ignore")
```

```
size_of_data= int(input("Enter size_of_data: "))
virtual_address= int(input("Enter virtual_address: "))
entropy= float(input("Enter entropy: "))
virtual_size= int(input("Enter virtual_size: "))


# Make predictions
input_data = [[size_of_data, virtual_address, entropy, virtual_size]]
# Load the correct file name
with open('pe_section_headers.pkl', 'rb') as file: # Changed file name
    loaded_model = pickle.load(file)

prediction = loaded_model.predict(input_data)
predicted_class = y_classes[prediction[0]]
print("Predicted target:", predicted_class)
```

```
Enter size_of_data: 230400
Enter virtual_address: 4096
Enter entropy: 6.392307
Enter virtual_size: 230342
Predicted target: 1
```

```
if(predicted_class==1):
  print("Malware")
else:
  print("Goodware")
```

```
Malware
```

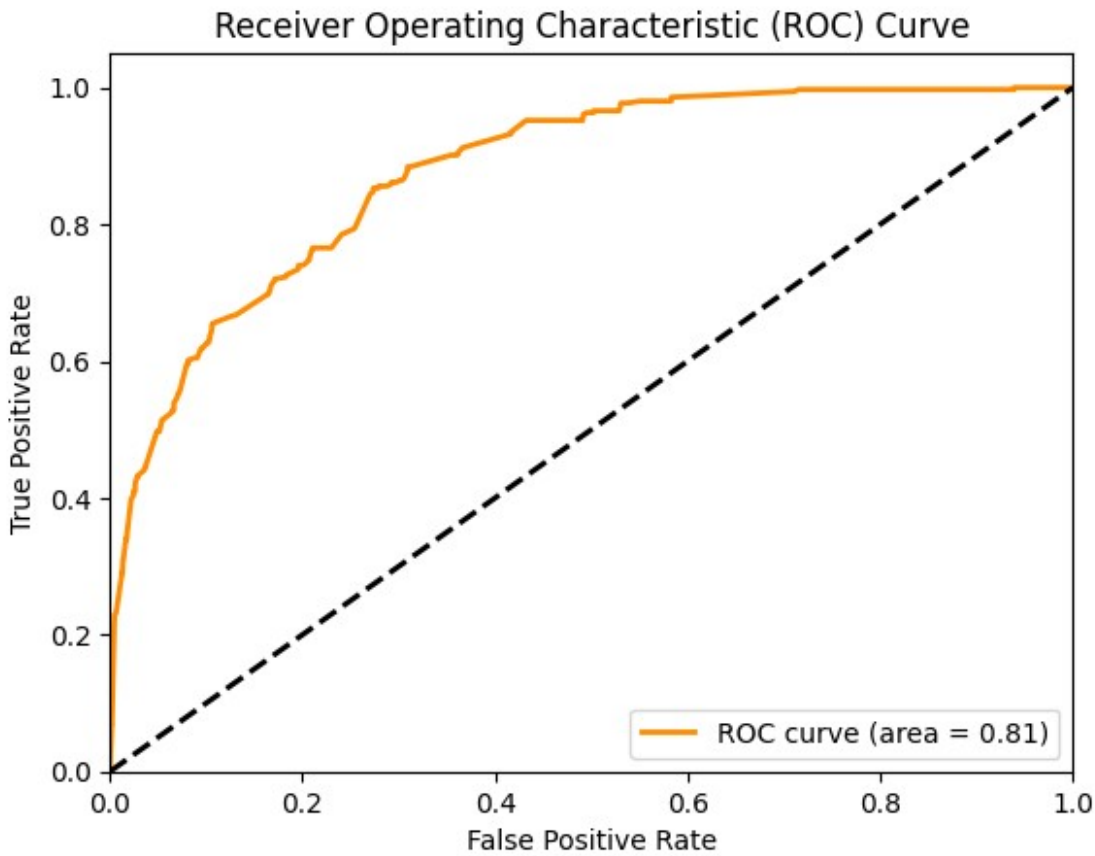
```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors = 8)
knn.fit(X_train , y_train)
```

```
KNeighborsClassifier(n_neighbors=8)
```

```
with open('pe_section_headers.pkl', 'wb') as file:
    pickle.dump(knn, file)

# Evaluate the model
y_pred = knn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

y_classes_str=y_classes.astype(str).tolist()
classification_metrics = classification_report(y_test, y_pred,
target_names=y_classes_str)
print("Classification Report:\n", classification_metrics)

conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)
```

```python
fpr, tpr, _ = roc_curve(y_test, y_proba[:, 1]) # Use the probabilities for the positive class
roc_auc = roc_auc_score(y_test, y_proba[:, 1])


# Calculate ROC and AUC
y_proba = knn.predict_proba(X_test)
#y_test_bin = label_binarize(y_test, classes=range(n_classes))
roc_auc = roc_auc_score(y_test, y_proba[:, 1], multi_class='ovr')
print("ROC AUC Score:", roc_auc)

# Plot ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:0.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

```
Accuracy: 0.9636216653193209
Classification Report:
              precision    recall  f1-score   support

           1       0.97      0.99      0.98      8305
           0       0.64      0.25      0.36       354

    accuracy                           0.96      8659
   macro avg       0.81      0.62      0.67      8659
weighted avg       0.96      0.96      0.96      8659

Confusion Matrix:
 [[8257   48]
 [ 267   87]]
ROC AUC Score: 0.8053298162906424
```




```
import warnings
warnings.filterwarnings("ignore")

size_of_data= int(input("Enter size_of_data: "))
virtual_address= int(input("Enter virtual_address: "))
entropy= float(input("Enter entropy: "))
virtual_size= int(input("Enter virtual_size: "))


# Make predictions
input_data = [[size_of_data, virtual_address, entropy, virtual_size]]
# Load the correct file name
with open('pe_section_headers.pkl', 'rb') as file: # Changed file name
    loaded_model = pickle.load(file)

prediction = loaded_model.predict(input_data)
predicted_class = y_classes[prediction[0]]
print("Predicted target:", predicted_class)

Enter size_of_data: 89088
Enter virtual_address: 4096
Enter entropy: 6.469704
```

```
Enter virtual_size: 88870
Predicted target: 1
```

```
if(predicted_class==1):
  print("Malware")
else:
  print("Goodware")
```

```
Malware
```

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(X_train , y_train)
```

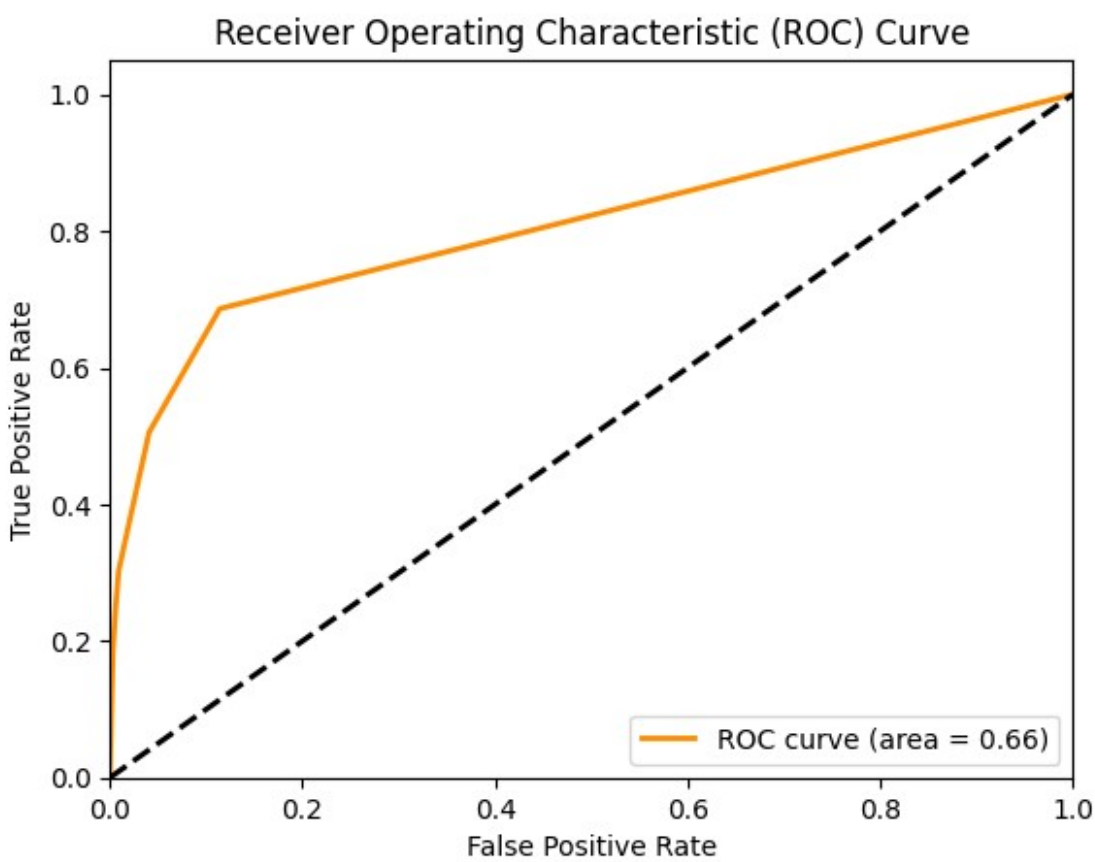
```
LogisticRegression()
```

```
with open('pe_section_headers.pkl', 'wb') as file:
    pickle.dump(lr, file)
```

```
# Evaluate the model
y_pred = lr.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

y_classes_str=y_classes.astype(str).tolist()
classification_metrics = classification_report(y_test, y_pred,
target_names=y_classes_str)
print("Classification Report:\n", classification_metrics)

conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)

fpr, tpr, _ = roc_curve(y_test, y_proba[:, 1]) # Use the probabilities
for the positive class
roc_auc = roc_auc_score(y_test, y_proba[:, 1])


# Calculate ROC and AUC
y_proba = lr.predict_proba(X_test)
#y_test_bin = label_binarize(y_test, classes=range(n_classes))
roc_auc = roc_auc_score(y_test, y_proba[:, 1], multi_class='ovr')
print("ROC AUC Score:", roc_auc)

# Plot ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area =
{roc_auc:0.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

```
Accuracy: 0.9591176810255225
Classification Report:
              precision    recall  f1-score   support

           1       0.96      1.00      0.98      8305
           0       0.00      0.00      0.00       354

    accuracy                           0.96      8659
   macro avg       0.48      0.50      0.49      8659
weighted avg       0.92      0.96      0.94      8659

Confusion Matrix:
 [[8305    0]
 [ 354    0]]
ROC AUC Score: 0.6590587659057746
```




```
import warnings
warnings.filterwarnings("ignore")
```

```python
size_of_data= int(input("Enter size_of_data: "))
virtual_address= int(input("Enter virtual_address: "))
entropy= float(input("Enter entropy: "))
virtual_size= int(input("Enter virtual_size: "))


# Make predictions
input_data = [[size_of_data, virtual_address, entropy, virtual_size]]
# Load the correct file name
with open('pe_section_headers.pkl', 'rb') as file: # Changed file name
    loaded_model = pickle.load(file)

prediction = loaded_model.predict(input_data)
predicted_class = y_classes[prediction[0]]
print("Predicted target:", predicted_class)
```

```
Enter size_of_data: 262144
Enter virtual_address: 4096
Enter entropy: 6.638142
Enter virtual_size: 261943
Predicted target: 1
```

```python
if(predicted_class==1):
  print("Malware")
else:
  print("Goodware")
```

```
Malware
```